

通常 Tello + Raspberry Pi ゲートウェイ構成 追加実装手順書

2026 年 6 月 10 日

1 目的

本手順書では、通常モデルの DJI Tello を複数台用いる協調取り囲み実験のために追加したソースコード群の導入手順をまとめる。通常 Tello は、各機体が Wi-Fi アクセスポイントとして動作し、Tello 側 IP アドレスが原則として各機体で同じ 192.168.10.1 になる。そのため、1 台の PC から複数の通常 Tello へ直接接続して制御する構成は取りにくい。

そこで、Tello の台数分の Raspberry Pi を用意し、各 Raspberry Pi を 1 台の Tello 専用ゲートウェイとして使う。デスクトップ PC は有線 LAN で各 Raspberry Pi と通信し、各 Raspberry Pi は Wi-Fi で対応する Tello に接続する。PC 側は外部計測、安全シールド、目標スロット計算、方策またはベースライン制御を担当し、Raspberry Pi 側は Tello SDK コマンド送信を担当する。

2 追加ファイル構成

追加したファイル構成は次の通りである。

```
tello_pi_gateway_patch/ |——
python/ |
  |—— run_real_baseline.py |
  |—— tello_rl/ |
    |—— real/ |
      |—— __init__.py |
      |—— pi_client.py |
      |—— real_tracker.py |
      |—— real_safety_shield.py |
      |—— real_tello_env.py |——
raspberry_pi/ |
  |—— tello_gateway.py |——
configs/ |
  |—— pc_real_config.example.json |
  |—— tracker_udp_schema.example.json |
  |—— pi_gateway_commands.txt |——
README_source_files.md
```

各ファイルの役割を表 1 に示す.

表 1: 追加ファイルの役割

ファイル	役割
<code>run_real_baseline.py</code>	実機構成で PD ベースライン制御を実行する PC 側エントリ. 学習済み方策を使う前の通信・外部計測・安全シールド確認に用いる.
<code>pi_client.py</code>	PC から各 Raspberry Pi の <code>tello_gateway.py</code> へ TCP/JSON Lines で指令を送るクライアント.
<code>real_tracker.py</code>	外部計測系から UDP JSON で送られてくる Tello と対象物の状態を受け取る.
<code>real_safety_shield.py</code>	実機用の安全シールド. 機体間距離, 壁距離, 高度範囲, 入力上限, 計測欠落を監視する.
<code>real_tello_env.py</code>	既存の <code>MockEnv/UnityEnv</code> と同様の API を持つ実機環境ラップ. 将来的に学習済み方策を実機へ接続するための入口.
<code>tello_gateway.py</code>	Raspberry Pi 上で実行する Tello SDK ゲートウェイ. PC から受け取った正規化行動を <code>rc a b c d</code> へ変換して Tello に送る.
<code>pc_real_config.example.json</code>	PC 側の実機構成設定例. Pi の IP, 飛行領域, 安全距離, 制御ゲインなどを定義する.
<code>tracker_udp_schema.example.json</code>	外部計測系が PC へ送る UDP JSON の形式例.
<code>pi_gateway_commands.txt</code>	Pi ゲートウェイに送れるメッセージ形式のメモ.

3 前提条件

本実装は, 以下の前提で動作する.

1. デスクトップ PC と全 Raspberry Pi は, 同じ有線 LAN に接続されている.
2. 各 Raspberry Pi は, `wlan0` で対応する 1 台の Tello の Wi-Fi に接続する.
3. PC から Tello の `192.168.10.1` へは直接アクセスしない. PC は各 Raspberry Pi の有線 LAN 側 IP アドレスだけを扱う.
4. 外部計測系が存在し, Tello の水平位置, 高度, `yaw`, 対象物位置を PC に送信できる.
5. 初期実験では, 学習済み方策ではなく PD ベースライン制御を用いる.
6. 実機飛行時は, 安全ネット, 十分な飛行領域, 緊急停止手段, 予備バッテリーを用意する.

4 クイックスタート：外部計測なしデモの準備手順

本節では、外部計測なしデモを実行する直前の準備手順を短くまとめる。詳細なネットワーク設定、SSH/SCP、Tello Wi-Fi 接続確認、および各デモの判定方法は後続の節で説明する。ここでは、3 台の Raspberry Pi の有線 LAN 側 IP を 192.168.50.101, 192.168.50.102, 192.168.50.103, 全 Raspberry Pi のユーザー名を inulab とする。

1. メイン PC の PowerShell から、各 Raspberry Pi へ SSH 接続する。gateway は各 Raspberry Pi 上で常駐させるため、Pi の台数分の SSH 画面を開いておく。

```
ssh inulab@192.168.50.101
ssh inulab@192.168.50.102
ssh inulab@192.168.50.103
```

各 SSH 画面で、Raspberry Pi 側の Python 仮想環境を有効化する。初回だけ `python3 -m venv /venv` を実行し、2 回目以降は `source /venv/bin/activate` だけでよい。

```
python3 -m venv ~/venv      # 初回のみ
source ~/venv/bin/activate
python -m pip install --upgrade pip
```

2. メイン PC 側の PowerShell で、Tello-drone/python に移動し、Python 仮想環境を有効化する。Activate.ps1 が実行ポリシーで拒否される場合は、初回のみ RemoteSigned を CurrentUser スcopeで許可する。

```
cd C:\Users\inulab\Work\Tello-drone\python
python -m venv venv
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

管理者権限や PC 全体の設定変更を避けたい場合は、その PowerShell セッションだけで次を実行してから仮想環境を有効化する。

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\venv\Scripts\Activate.ps1
```

3. 3 台の Tello の電源を入れる。各 Raspberry Pi が対応する Tello Wi-Fi へ接続するまで少し待つ。各 SSH 画面で次を確認し、TELLO-XXXXXX と 192.168.10.1 への経路が見えることを確認する。

```
iwgetid
ip route get 192.168.10.1
ping -c 3 192.168.10.1
```

4. 各 Raspberry Pi の SSH 画面で gateway を起動する。Wi-Fi 接続が完了していない状態で起動すると `OSError: [Errno 101] Network is unreachable` になることがある。その場

合は、`iwgetid` と `ip route get 192.168.10.1` を確認し、Tello Wi-Fi へ接続できてから同じコマンドを再実行する。

```
source ~/venv/bin/activate
python3 tello_gateway.py --listen-host 0.0.0.0 --listen-port 10000 --auto-command
```

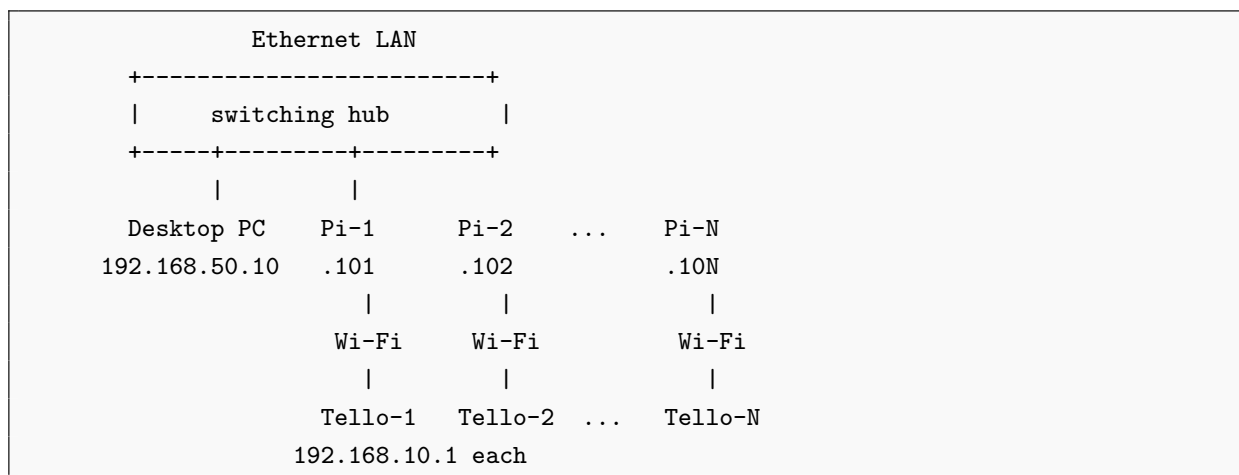
gateway が起動している SSH 画面ではプロンプトが戻らない。その画面は閉じずに残し、デモスクリプトはメイン PC 側の別 PowerShell から実行する。

5. メイン PC 側でデモスクリプトを実行する。まず非飛行の Demo 1 で通信を確認し、その後 Demo 2 の短時間 takeoff-hover-land へ進む。

```
cd C:\Users\inulab\Work\Tello-drone\python
.\venv\Scripts\Activate.ps1
python .\no_tracker_demos\demo_01_comm_check.py --config ..\configs\
    no_tracker_demo_config.local.json --repeat 3
python .\no_tracker_demos\demo_02_takeoff_hover_land.py --config ..\configs\
    no_tracker_demo_config.local.json --duration 8 --rate-hz 10 --yes
```

5 ネットワーク構成

推奨するネットワーク構成を以下に示す。



有線 LAN 側 IP アドレスの例を表 2 に示す。

表 2 有線 LAN 側 IP アドレス設定例

機器	インターフェース	IP アドレス
デスクトップ PC	Ethernet	192.168.50.10/24
Raspberry Pi 1	eth0	192.168.50.101/24
Raspberry Pi 2	eth0	192.168.50.102/24
Raspberry Pi 3	eth0	192.168.50.103/24

Raspberry Pi は IP ルータとしては使わない。したがって、IP forwarding や bridge 設定は不要である。Raspberry Pi は PC と Tello の間でアプリケーション層の変換を行うだけである。

6 Raspberry Pi 側の準備

6.1 ファイル配置

各 Raspberry Pi に、次のファイルをコピーする。

```
raspberrypi/tello_gateway.py
```

例えば PC から Raspberry Pi へコピーする場合は次のようにする。

```
scp raspberrypi/tello_gateway.py inulab@192.168.50.101:~/tello_gateway.py
```

Pi が複数台ある場合は、各 Pi に同じ tello_gateway.py をコピーする。Pi ごとの差分は、起動時の引数または接続先 Tello Wi-Fi の SSID で管理する。

6.2 Raspberry Pi 側 Python 仮想環境の準備

各 Raspberry Pi では、gateway 実行用の Python 仮想環境を用意してから tello_gateway.py を起動する。仮想環境の作成は初回のみでよい。

```
python3 -m venv ~/venv
source ~/venv/bin/activate
python -m pip install --upgrade pip
```

以降、SSH 接続し直した場合は、gateway を起動する前に次を実行して仮想環境を有効化する。

```
source ~/venv/bin/activate
```

6.3 有線 LAN 側 IP の固定

Raspberry Pi OS で NetworkManager を用いる場合、Pi-1 では次のように有線 LAN 側 IP を固定する。

```
sudo nmcli con mod "Wired connection 1" ipv4.addresses 192.168.50.101/24
sudo nmcli con mod "Wired connection 1" ipv4.method manual
sudo nmcli con up "Wired connection 1"
```

Pi-2 では 192.168.50.102/24、Pi-3 では 192.168.50.103/24 のように割り当てる。

6.4 Tello Wi-Fi への接続

各 Raspberry Pi の wlan0 を、対応する Tello の Wi-Fi に接続する。

```
sudo nmcli dev wifi rescan
nmcli dev wifi list
sudo nmcli dev wifi connect TELLO-XXXXXX ifname wlan0
```

接続後, Pi から Tello に到達できるか確認する.

```
ping 192.168.10.1
```

6.5 Pi ゲートウェイの起動

各 Raspberry Pi で `tello_gateway.py` を起動する. 初期確認では, `--auto-command` を付けて起動時に Tello SDK mode へ入るようにする.

```
python3 ~/tello_gateway.py \  
--listen-host 0.0.0.0 \  
--listen-port 10000 \  
--auto-command \  
--rc-default-limit 30
```

安全確認中は `--rc-default-limit` を 20 から 30 程度に制限する. いきなり 100 にしない.

7 PC から Raspberry Pi へ SSH/SCP するためのネットワーク設定

本節では, PC から Raspberry Pi を遠隔操作するための SSH/SCP 設定をまとめる. 本構成では, Raspberry Pi は Tello と Wi-Fi で 1 対 1 接続しつつ, PC とはスイッチングハブ経由の有線 LAN で接続される. したがって, PC から Raspberry Pi へ SSH する際は, Tello 側の Wi-Fi アドレスではなく, Raspberry Pi の有線 LAN 側 IP アドレスを指定する.

7.1 二つのネットワークを分離する考え方

Raspberry Pi は次の二つのネットワークに同時に参加する.

```
eth0   または end0 : PC との制御・SSHSCP 用  
wlan0           : Tello との SDK 通信用
```

たとえば Pi-1 では次のように設定する.

```
PC Ethernet : 192.168.50.10/24  
Pi eth0     : 192.168.50.101/24  
Pi wlan0    : 192.168.10.x/24  
Tello      : 192.168.10.1
```

有線 LAN 側と Tello Wi-Fi 側を同じ 192.168.10.0/24 にしてはいけない. Tello は各機体で同じ 192.168.10.1 を使うため, 有線 LAN 側は 192.168.50.0/24 などの別ネットワークに分ける.

7.2 PC 側 Ethernet の固定 IP 設定

Windows PC 側では, スイッチングハブに接続している Ethernet アダプタに固定 IP を設定する.

```
アドレス  
IP       : 192.168.50.10サブネットマスク
```

```
 : 255.255.255.0デフォルトゲートウェイ
 : 空欄
DNS : 空欄
```

スイッチングハブだけでは DHCP により IP アドレスは配布されないため、PC と Raspberry Pi の両方に固定 IP を設定する。PC 側の設定確認は PowerShell で次を実行する。

```
ipconfig
```

7.3 Raspberry Pi 側有線 LAN の固定 IP 設定

Raspberry Pi 側では、有線 LAN の接続名を確認する。Raspberry Pi の機種や OS により、インターフェース名は `eth0` ではなく `end0` になる場合がある。

```
nmcli con show
ip -br addr
```

有線接続名が `Wired connection 1` の場合、Pi-1 では次のように設定する。

```
sudo nmcli con mod "Wired connection 1" \
  ipv4.method manual \
  ipv4.addresses 192.168.50.101/24 \
  ipv4.gateway "" \
  ipv4.dns "" \
  ipv6.method disabled

sudo nmcli con up "Wired connection 1"
```

Pi-2 では `192.168.50.102/24`、Pi-3 では `192.168.50.103/24` のように、Pi ごとに異なる IP アドレスを割り当てる。設定後、次のような状態になっていることを確認する。

```
ip -br addr

# 期待例
eth0    UP    192.168.50.101/24
wlan0   UP    192.168.10.x/24
```

または、Raspberry Pi 5 などでは次のように表示されることがある。

```
end0    UP    192.168.50.101/24
wlan0   UP    192.168.10.x/24
```

7.4 Tello Wi-Fi 接続時の注意

Raspberry Pi の Wi-Fi 側は、対応する Tello の SSID に接続する。このとき、Tello Wi-Fi をデフォルト経路として使わないようにするため、`ipv4.never-default yes` を設定しておくといよい。

```
sudo nmcli dev wifi rescan
```

```
nmcli dev wifi list
```

```
sudo nmcli con add type wifi ifname wlan0 con-name tello1 ssid TELLO-XXXXXX
sudo nmcli con mod tello1 ipv4.method auto ipv4.never-default yes ipv6.method disabled
sudo nmcli con up tello1
```

接続後, Raspberry Pi から Tello に到達できることを確認する.

```
ping 192.168.10.1
```

7.5 SSH サーバの有効化

PC から SSH 接続するには, Raspberry Pi 側で SSH サーバが起動している必要がある. Raspberry Pi 上で次を実行する.

```
sudo systemctl enable --now ssh
sudo systemctl status ssh
```

SSH サーバが入っていない場合は, 次でインストールする.

```
sudo apt update
sudo apt install -y openssh-server
sudo systemctl enable --now ssh
```

22 番ポートで待ち受けているかは次で確認できる.

```
sudo ss -tlnp | grep :22
```

7.6 PC から SSH 接続する

PC の PowerShell から, 有線 LAN 側 IP アドレスを指定して接続する. 本実験で用いる Raspberry Pi のユーザー名は全台で inulab に統一しているため, SSH/SCP では inulab@<Pi の有線 LAN 側 IP> を指定する.

```
ssh inulab@192.168.50.101
ssh inulab@192.168.50.102
ssh inulab@192.168.50.103
```

ここで, 192.168.50.101, 192.168.50.102, 192.168.50.103 はそれぞれ Pi-1, Pi-2, Pi-3 の有線 LAN 側 IP アドレスである. プロンプトが inulab@inulab-rpi-01: \$ のように表示される場合, @ の前の inulab がユーザー名, @ の後の inulab-rpi-01 がホスト名である.

ユーザー名を確認したい場合, Raspberry Pi にログイン済みの端末で次を実行する.

```
whoami
```

本構成では, ここで inulab と表示されることを確認する.

7.7 SCP によるファイル転送

scp は SSH 経由でファイルを転送するコマンドである。PC から Raspberry Pi へファイルを送る場合は、PC 側の PowerShell で実行する。Raspberry Pi に SSH ログインした状態で Windows の C: ドライブを指定しても、Raspberry Pi から Windows のローカルファイルは見えない。

PC から Pi-1 へ tello_gateway.py を送る例を示す。

```
scp .\tello_pi_gateway_patch\raspberry_pi\tello_gateway.py \
inulab@192.168.50.101:~/tello_gateway.py
```

絶対パスで指定する場合は次のようにする。

```
scp "C:\Users\inulab\Work\Tello-drone\tello_pi_gateway_patch\raspberry_pi\tello_gateway.
py" \
inulab@192.168.50.101:~/tello_gateway.py
```

転送後、SSH で Raspberry Pi に入り、ファイルが存在するか確認する。

```
ssh inulab@192.168.50.101 "ls -l ~/tello_gateway.py"
```

Raspberry Pi から PC へログを回収する場合は、PC 側 PowerShell で次のように実行する。

```
scp inulab@192.168.50.101:/home/inulab/log.csv .\
```

7.8 SSH/SCP の典型的なエラーと対処

7.8.1 ping がタイムアウトする

ping 192.168.50.101 がタイムアウトする場合は、SSH 以前に有線 LAN 通信が成立していない。次を確認する。

- PC の Ethernet IP が 192.168.50.10/24 になっているか。
- Raspberry Pi の有線 LAN IP が 192.168.50.101/24 になっているか。
- PC と Pi が同じスイッチングハブに接続されているか。
- LAN ケーブルまたはハブのリンクランプが点灯しているか。
- Pi 側で ip link を実行したとき、有線インターフェースが NO-CARRIER になっていないか。

Pi 側の物理リンクは次で確認する。

```
ip link
```

LOWER_UP が表示されていれば物理リンクは成立している。NO-CARRIER の場合は、ケーブル、ハブ、ポート、電源を確認する。

7.8.2 Connection refused

ssh: connect to host 192.168.50.101 port 22: Connection refused と表示される場合、IP には到達しているが、Raspberry Pi 側で SSH サーバが起動していない可能性が高い。Pi 側で次を実行する。

```
sudo systemctl enable --now ssh
sudo systemctl status ssh
```

7.8.3 Could not resolve hostname c

Raspberry Pi に SSH ログインした状態で次のようなコマンドを実行すると, C:/Users/... の C: がリモートホスト指定と解釈され, Could not resolve hostname c になる.

```
scp "C:/Users/inulab/Work/.../tello_gateway.py" \
inulab@192.168.50.101:~/tello_gateway.py
```

PC 上のファイルを Pi に送る scp は, Raspberry Pi 上ではなく, Windows PowerShell 上で実行する.

7.8.4 Permission denied

scp または ssh で Permission denied になる場合は, ユーザー名またはパスワードが違う可能性が高い. 本実験では Raspberry Pi のユーザー名を inulab に統一しているため, 次の形式で接続する.

```
ssh inulab@192.168.50.101
scp .\tello_pi_gateway_patch\raspberry_pi\tello_gateway.py \
inulab@192.168.50.101:~/tello_gateway.py
```

プロンプトが次のように表示される場合, inulab がユーザー名, inulab-rpi-01 がホスト名である. ホスト名をユーザー名として指定しないように注意する.

```
inulab@inulab-rpi-01:~ $
```

7.9 推奨する確認順序

作業時は次の順に確認すると, 問題を切り分けやすい.

1. PC の Ethernet IP が 192.168.50.10/24 であることを確認する.
2. Raspberry Pi の有線 LAN IP が 192.168.50.101/24 であることを確認する.
3. Raspberry Pi の Wi-Fi が Tello に接続され, wlan0 に 192.168.10.x/24 が付いていることを確認する.
4. PC から ping 192.168.50.101 が通ることを確認する.
5. Raspberry Pi で SSH サーバが起動していることを確認する.
6. PC から ssh inulab@192.168.50.101 でログインする.
7. PC の PowerShell から scp で必要なファイルを転送する.
8. Raspberry Pi 上で ls -l により転送されたファイルを確認する.

8 PC 側の準備

8.1 ファイル配置

既存プロジェクトの Python ディレクトリに、追加ファイルをコピーする。既存構成が次のような場合を想定する。

```
project_root/ └──
python/
  ├── train.py
  └── tello_rl/
      ├── config.py
      ├── formation.py
      ├── observation.py
      └── ...
```

ここに追加実装を次のように配置する。

```
project_root/ └──
python/
  ├── run_real_baseline.py
  └── tello_rl/
      └── real/
          ├── __init__.py
          ├── pi_client.py
          ├── real_tracker.py
          ├── real_safety_shield.py
          └── real_tello_env.py
```

設定ファイルは、任意の場所に置いてよいが、以下では `project_root/configs/` に置く例とする。

```
project_root/ └──
configs/
  ├── pc_real_config.example.json
  ├── tracker_udp_schema.example.json
  └── pi_gateway_commands.txt
```

8.2 Python 仮想環境と PowerShell 実行ポリシー

メイン PC 側では、既存プロジェクトの `python/` ディレクトリで仮想環境を作成し、有効化してからデモスクリプトを実行する。Windows PowerShell で `Activate.ps1` の実行が拒否される場合は、ユーザー単位で `RemoteSigned` を許可する。

```
cd C:\Users\inulab\Work\Tello-drone\python
python -m venv venv
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
.\venv\Scripts\Activate.ps1
```

```
python -m pip install --upgrade pip
```

すでに仮想環境を作成済みの場合、通常は次だけでよい。PowerShell のプロンプト先頭に (venv) が付けば有効化できている。

```
cd C:\Users\inulab\Work\Tello-drone\python
.\venv\Scripts\Activate.ps1
```

PC の実行ポリシーを恒久的に変更したくない場合は、現在の PowerShell セッションだけで一時的に許可してから有効化する。

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\venv\Scripts\Activate.ps1
```

8.3 Python パスの確認

PC 側では、既存プロジェクトの python/ ディレクトリで実行する。

```
cd project_root/python
python -c "import tello_rl; print('ok')"
python -c "from tello_rl.real import PiSwarmClient; print('real ok!')"
```

インポートに失敗する場合は、実行ディレクトリまたは PYTHONPATH を確認する。

```
# 例: project_root/python を PYTHONPATH に追加
export PYTHONPATH=$(pwd):$PYTHONPATH
```

Windows PowerShell の場合は次のようにする。

```
$env:PYTHONPATH = (Get-Location).Path + ";" + $env:PYTHONPATH
```

9 PC 側設定ファイル

pc_real_config.example.json をコピーし、実験用に編集する。

```
cp ../configs/pc_real_config.example.json ../configs/pc_real_config.local.json
```

主に編集する項目は次である。

表 3: PC 側設定ファイルの主な項目

項目	意味
N	Tello の台数。初期実験では 1, 次に 2, 最後に 3 と段階的に増やす。
dt	制御周期。初期値は 0.1 秒程度。実機では 5–10 Hz から始める。

項目	意味
pi_gateways	各 Raspberry Pi の名前, IP, ポートを列挙する.
tracker	外部計測 UDP を受信するホストとポート.
area	飛行領域の x, y, z 範囲. 実験室の実寸に合わせる.
safety	機体間距離, 壁距離, 高度範囲, 入力上限, 計測タイムアウトなどの安全設定.
controller	PD ベースラインのゲインや速度上限.

Pi の IP アドレスを実機環境に合わせて修正する. 例を以下に示す.

```
"pi_gateways": [
  {"name": "tello_1", "host": "192.168.50.101", "port": 10000},
  {"name": "tello_2", "host": "192.168.50.102", "port": 10000},
  {"name": "tello_3", "host": "192.168.50.103", "port": 10000}
]
```

10 外部計測 UDP の形式

PC 側の `real_tracker.py` は, 外部計測系から UDP JSON を受け取る. 送信する JSON の基本形式は次の通りである.

```
{
  "t": 12.345,
  "target": {
    "r": [0.0, 0.0],
    "v": [0.0, 0.0]
  },
  "drones": [
    {"id": 0, "r": [0.9, 0.0], "h": 1.0, "yaw": 3.14},
    {"id": 1, "r": [-0.45, 0.78], "h": 1.0, "yaw": -1.05},
    {"id": 2, "r": [-0.45, -0.78], "h": 1.0, "yaw": 1.05}
  ]
}
```

ここで, r は水平面座標 $[x, y]$, h は高度, yaw はラジアン単位の yaw 角である. Unity 座標系や画像座標系から変換する場合は, PC 側へ送る前に実験座標系へ変換しておく.

11 実装後の確認手順

11.1 Step 0: 構文チェック

まず, PC 側と Raspberry Pi 側の Python ファイルに構文エラーがないか確認する.

```
# PC 側
cd project_root/python
```

```
python -m py_compile run_real_baseline.py
python -m py_compile tello_rl/real/pi_client.py
python -m py_compile tello_rl/real/real_tracker.py
python -m py_compile tello_rl/real/real_safety_shield.py
python -m py_compile tello_rl/real/real_tello_env.py

# Raspberry Pi 側
python3 -m py_compile ~/tello_gateway.py
```

11.2 Step 1: Pi 単体で Tello SDK 通信確認

各 Pi で Tello Wi-Fi に接続し、ゲートウェイを起動する。PC から接続する前に、Pi 上で Tello に command が送られていること、バッテリー問い合わせができることを確認する。

ゲートウェイが PC から query メッセージを受けられる場合、PC から次のようなメッセージを送る。

```
{"type": "query", "cmd": "battery?"}
```

応答が返れば、Pi-Tello 間の UDP 通信は成立している。

11.3 Step 2: PC から Pi への接続確認

PC から各 Raspberry Pi へ TCP 接続できるか確認する。

```
ping 192.168.50.101
ping 192.168.50.102
ping 192.168.50.103
```

次に、PC 側プログラムから各 Pi へ rc 0 0 0 0 相当のメッセージを送り、ゲートウェイ側ログに受信が出ることを確認する。この段階では、Tello を離陸させない。

11.4 Step 3: 外部計測のみの確認

Tello を飛ばさず、外部計測系が UDP JSON を PC へ送信しているか確認する。PC 側の real_tracker.py が受信できる形式になっていることを確認する。

チェックすべき点は次である。

- Tello の個体 ID と drones 配列の ID が一致している。
- 単位が m と rad になっている。
- 座標系の x, y 軸向きが、制御で使う座標系と一致している。
- yaw 角の正方向が、Tello の前方方向と一致している。
- UDP 更新周期が制御周期より十分速いか、少なくとも同程度である。

11.5 Step 4: PC 側ベースラインを dry run する

--takeoff を付けずに PC 側ベースラインを実行する。この段階では、制御計算、外部計測受信、Pi への接続だけを確認する。

```
cd project_root/python
python run_real_baseline.py \
  --config ../configs/pc_real_config.local.json \
  --duration 20
```

ログ上で、次を確認する。

- 各 Tello の計測値が更新されている。
- 目標スロットが飛行領域内にある。
- 安全シールドが異常な補正を連発していない。
- Pi への送信が成功している。

11.6 Step 5: 単機 takeoff/land 確認

最初は $N = 1$ に設定し、Tello 1 台だけで確認する。外部計測が正常に入っている状態で、PC 側から takeoff と land を行う。

```
python run_real_baseline.py \
  --config ../configs/pc_real_config.local.json \
  --duration 10 \
  --takeoff
```

この段階では、制御入力を小さくし、必要であれば目標位置を現在位置付近に置く。正常に離陸し、ホバリングし、着陸できることを確認する。

11.7 Step 6: 単機位置制御

次に、単機で目標位置への PD 制御を確認する。初期設定では、速度上限と RC 上限を低くする。

- rc_limit: 20–30
- 水平速度上限: 0.2–0.3 m/s
- 高度目標: 0.8–1.0 m
- 制御周期: 0.1–0.2 s

機体が目標位置と逆方向に動く場合は、座標系または yaw 角の符号が合っていない。左右、前後、yaw の符号を単機で再確認する。

11.8 Step 7: 複数機の固定位置制御

単機で安定した後、 $N = 2$ 、次に $N = 3$ へ増やす。この段階では対象物取り囲みではなく、各機体に十分離れた固定目標位置を与える。安全距離を大きめに設定し、機体間距離が保たれることを確認する。

11.9 Step 8: 静止対象物の正多角形取り囲み

$N = 3$ 、対象物静止、Case 1 の正多角形配置を用いる。取り囲み半径 R は 0.9 m 程度から始める。初期実験では、対象物を床上の固定マーカーとし、外部計測系が対象物位置 r_o を送信する。評価するログ項目は次である。

- slot 誤差 $\|r_i - r_i^{\text{slot}}\|$
- 対象物からの半径誤差
- 機体間距離
- 壁距離
- 高度誤差
- safety shield の補正回数
- Pi との通信遅延

11.10 Step 9: 移動対象物追従

静止対象物で安定してから、対象物を低速で動かす。対象物速度は最初は 0.05–0.1 m/s 程度に制限する。急な折れ線軌道や円軌道は、静止対象物と等速直線追従が安定してから行う。

11.11 Step 10: 学習済み方策への置き換え

PD または NSB ベースラインで安全性を確認した後、学習済み方策を RealTelloEnv に接続する。このときも、安全シールドは無効化しない。最初は探索ノイズを切り、方策の平均 action のみを用いる。RC 上限も低い値から始める。

12 安全確認チェックリスト

実機飛行前に、以下を確認する。

1. 各 Raspberry Pi から対応する Tello にだけ接続されている。
2. PC から各 Raspberry Pi に ping が通る。
3. 外部計測系が全 Tello と対象物を認識している。
4. Tello ID と Pi ID と外部計測 ID の対応が一致している。
5. 座標系の x, y 軸, yaw 正方向, Tello 前方方向が一致している。
6. PC 側安全シールドが有効である。

7. Pi 側 watchdog が有効である.
8. PC 側プログラムを停止した場合に, Pi が `rc 0 0 0 0` または `land` を送る.
9. `emergency` を送る方法が確認されている.
10. バッテリー残量が十分である.
11. 飛行領域に人や壊れやすい物がない.
12. 初回は必ず低い RC 上限で実行する.

13 トラブルシューティング

13.1 PC から Pi に接続できない

次を確認する.

- Pi の有線 LAN 側 IP が設定通りか.
- PC と Pi が同じサブネットにいるか.
- `tello_gateway.py` が起動しているか.
- ファイアウォールが TCP ポート 10000 をブロックしていないか.

13.2 Pi から Tello に接続できない

次を確認する.

- Pi の `wlan0` が正しい Tello SSID に接続されているか.
- `ping 192.168.10.1` が通るか.
- 他の端末が同じ Tello Wi-Fi に接続していないか.
- Tello のバッテリーが十分か.

13.3 Tello が逆方向に動く

座標系または符号規約の問題である可能性が高い. 単機で次を確認する.

- `rc 0 20 0 0` で機体が前方へ動くか.
- `rc 20 0 0 0` で左右のどちらへ動くか.
- `yaw` 角が増える方向と画像処理で推定した `yaw` の正方向が一致しているか.
- 世界座標速度から機体座標速度への変換式が正しいか.

13.4 安全シールドが常に停止を返す

次を確認する.

- 外部計測の座標単位が `cm` ではなく `m` になっているか.

- 飛行領域設定 `area` が実験室座標と一致しているか.
- 高度 h の値が正しく入っているか.
- Tello ID と計測 ID が入れ替わっていないか.
- `tracker timeout` が短すぎないか.

14 実験ログの保存項目

実機実験では、既存の目的コスト・制約コスト・取り囲み誤差に加えて、通信系と安全系のログを保存する．最低限保存すべき項目は次である．

- 制御ステップ k と時刻 t .
- 各 Tello の位置 r_i , 高度 h_i , yaw 角 ψ_i .
- 対象物位置 r_o , 対象物速度 v_o .
- 目標スロット r_i^{slot} .
- raw action $\tilde{a}_i$.
- executed action a_i .
- 実際に送信した RC 値.
- 安全シールドが action を補正した理由.
- 各 Pi への送信成功・失敗.
- PC-Pi 間通信遅延.
- Tello のバッテリー残量.
- 外部計測の最終更新時刻.
- watchdog 発火の有無.

15 既存学習コードとの接続方針

追加実装のうち、`real_tello_env.py` は、既存の `MockEnv` や `UnityEnv` と同じ形式の API を提供することを目的としている．すなわち、外部からは次のように扱う．

```
obs = env.reset()
obs, obj, con, exe, done, info = env.step(raw_actions)
```

ただし、実機環境では、`step` の中で数値モデルを前進させるのではなく、次を行う．

1. raw action を安全シールドに通す.
2. executed action を各 Raspberry Pi へ送る.
3. 次の制御周期まで待つ.
4. 外部計測から実状態を取得する.
5. 観測, 目的コスト, 制約コストを計算して返す.

このため、学習済み方策の実機評価は可能であるが、実機上でオンライン学習を行う場合は、安全性、試行回数、バッテリー、計測欠落時の扱いを別途慎重に設計する必要がある．初期段階では、シミュ

レーションまたは Unity で学習した方策を、実機では評価のみ行う構成が望ましい。

16 外部計測なしで行う Tello–Raspberry Pi 協調制御基盤確認デモ

この節では、外部計測系を導入する前の段階で、3 台の通常 Tello, 3 台の Raspberry Pi, およびメイン PC が通信し、協調制御用の指令配信基盤として動作することを確認するデモ手順をまとめる。本デモは、対象物取り囲み制御そのものの検証ではなく、通信、周期指令、役割別 action 配信, watchdog, ログ保存を確認するための準備段階である。

16.1 目的

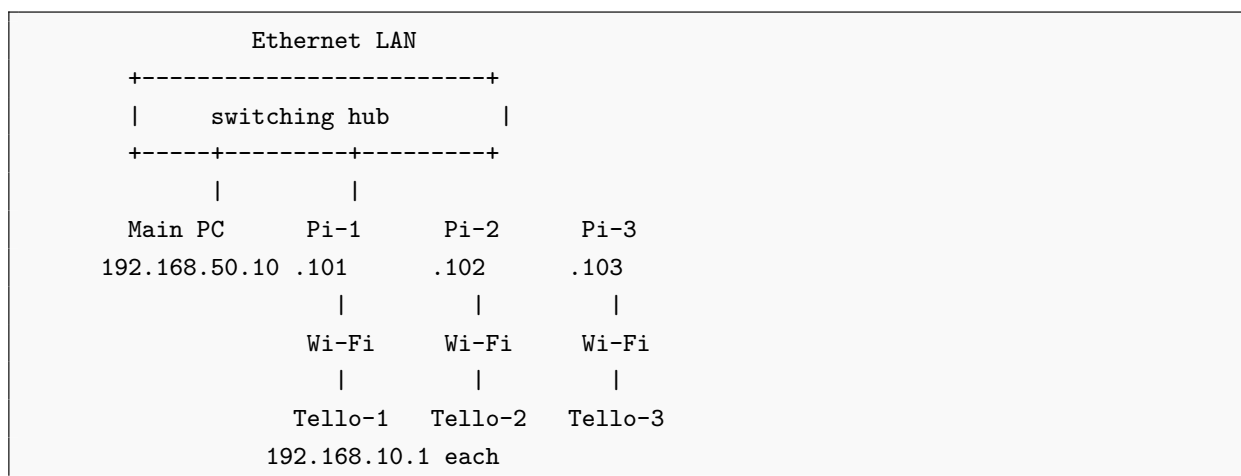
本書は、外部計測系を導入する前の段階で、3 台の通常 Tello, 3 台の Raspberry Pi, およびメイン PC が通信し、協調制御用の指令配信基盤として動作することを確認するためのデモ手順をまとめる。

ここで確認するのは、対象物取り囲み制御そのものではない。外部計測なしでは、各 Tello の水平面位置、機体間距離、対象物位置を十分な精度で得られないため、取り囲み誤差や安全距離維持を実証することはできない。本書のデモは、以下を確認するための準備段階である。

- メイン PC から 3 台の Raspberry Pi へ TCP 接続できること。
- 各 Raspberry Pi が対応する Tello に SDK コマンドを送れること。
- 3 台分の Tello から ok, battery? 応答、状態値を取得できること。
- メイン PC が 3 台へ周期的に rc 指令を送れること。
- 3 台に異なる役割を割り当て、yaw のみで安全寄りの同期動作を実行できること。
- 仮想的な合意計算・イベントトリガ計算の結果を、実機への役割別コマンドに反映できること。
- 各ステップの通信ログを JSONL 形式で保存できること。

16.2 前提構成

本デモでは、通常 Tello の通信制約を避けるため、各 Tello に 1 台ずつ Raspberry Pi を対応させる。各 Raspberry Pi は、有線 LAN 側でメイン PC と通信し、無線 LAN 側で対応する Tello の Wi-Fi に接続する。



重要な点は、Tello 側の 192.168.10.1 をメイン PC 側へルーティングしないことである。メイン PC は、各 Raspberry Pi の有線 LAN 側 IP アドレスだけを相手にし、Raspberry Pi がアプリケーション層ゲートウェイとして Tello SDK コマンドを送信する。

16.3 追加ファイル構成

本書に対応する追加ファイルは次の通りである。Raspberry Pi 側は、既に作成した `tello_gateway.py` をそのまま使う。今回追加するのは主にメイン PC 側のデモスクリプトである。

```
tello_no_tracker_demo_patch/  
python/  
  no_tracker_demos/  
    no_tracker_common.py  
    demo_01_comm_check.py  
    demo_02_takeoff_hover_land.py  
    demo_03_sync_yaw_roles.py  
    demo_04_virtual_event_demo.py  
configs/  
  no_tracker_demo_config.example.json  
  run_commands.md  
docs/  
  tello_no_tracker_demo_steps.tex  
README_no_tracker_demos.md
```

各ファイルの役割を表 4 に示す。

表 4: 外部計測なしデモ用ファイル

ファイル	役割
<code>no_tracker_common.py</code>	各デモ共通の TCP クライアント, JSONL ロガー, 並列送信, 安全確認, <code>land</code> 処理をまとめた共通モジュール。
<code>demo_01_comm_check.py</code>	非飛行の通信確認デモ。 <code>command</code> , <code>battery?</code> , <code>status</code> を 3 台同時に確認する。
<code>demo_02_takeoff_hover_land.py</code>	3 機同時の <code>takeoff</code> , 周期的な <code>rc 0 0 0 0</code> , <code>land</code> を確認する。水平移動は行わない。
<code>demo_03_sync_yaw_roles.py</code>	3 機に異なる <code>yaw</code> 役割を与える同期デモ。水平移動なしで, 役割別 <code>action</code> 配信を確認する。
<code>demo_04_virtual_event_demo.py</code>	メイン PC 内で仮想合意・イベントトリガ計算を行い, イベント発火を <code>yaw pulse</code> として反映するデモ。デフォルトでは非飛行 <code>dry-run</code> 。
<code>no_tracker_demo_config.example.json</code>	Raspberry Pi の IP アドレス, ポート, RC 上限, ログ保存先の設定例。

ファイル	役割
run_commands.md	代表的な実行コマンド一覧.

16.4 Raspberry Pi 側の準備

各 Raspberry Pi で、対応する Tello の Wi-Fi に接続してから、前回作成したゲートウェイを起動する。例を以下に示す。

```
python3 tello_gateway.py \
  --listen-host 0.0.0.0 \
  --listen-port 10000 \
  --auto-command \
  --rc-default-limit 25
```

各 Raspberry Pi の有線 LAN 側 IP は、設定ファイル `no_tracker_demo_config.example.json` と一致させる。例を以下に示す。

```
Pi-1: 192.168.50.101
Pi-2: 192.168.50.102
Pi-3: 192.168.50.103
```

Raspberry Pi 側では、メイン PC から一定時間 `rc` 指令が来ない場合、`watchdog` により `rc 0 0 0` または `land` を送る。そのため、飛行デモ中はメイン PC 側が周期的に `rc` を送り続ける。

16.5 メイン PC 側の設定

メイン PC 側では、`configs/no_tracker_demo_config.example.json` を実験環境に合わせて編集する。

```
{
  "gateways": [
    {"name": "tello-1", "host": "192.168.50.101", "port": 10000},
    {"name": "tello-2", "host": "192.168.50.102", "port": 10000},
    {"name": "tello-3", "host": "192.168.50.103", "port": 10000}
  ],
  "pi_timeout_s": 1.0,
  "rc_limit": 25,
  "rate_hz": 10.0,
  "log_dir": "logs/no_tracker_demos"
}
```

最初は `rc_limit=25` 程度の低い値から始める。本書の飛行デモでは水平移動成分 `lr`, `fb` は常に 0 とし、必要な場合も `yaw` のみを使う。

16.6 安全上の注意

外部計測なしでは、機体間距離と壁距離を自動的に監視できない。したがって、本デモでは次の条件を満たす場合だけ飛行デモを行う。

- プロペラガードを装着する。
- 3 台の Tello を十分に離して配置する。
- 水平移動を行わない。
- 最初は takeoff-hover-land のみを行う。
- yaw デモでは yaw_norm と rc_limit を小さくする。
- 飛行領域に人の手や顔を近づけない。
- 異常時はすぐにプログラムを停止し、必要なら全機 land または emergency を送る。

本デモの Python スクリプトは、飛行を伴う場合、通常は対話的に YES 入力を求める。自動実行したい場合だけ --yes を付ける。

16.7 Demo 1: 非飛行・通信確認デモ

16.7.1 目的

Demo 1 は、プロペラを回さずに、以下の通信経路を確認する。

```
Main PC -> Pi-i -> Tello-i -> Pi-i -> Main PC
```

確認項目は次の通りである。

- メイン PC から各 Raspberry Pi に TCP 接続できる。
- 各 Raspberry Pi が対応する Tello に command を送れる。
- 各 Tello から battery? 応答が返る。
- 各 Raspberry Pi の gateway 状態を status で取得できる。
- 3 台分の応答時間と結果を JSONL ログに保存できる。

16.7.2 実行コマンド

```
cd tello_no_tracker_demo_patch/python/no_tracker_demos
python demo_01_comm_check.py \
  --config ../../configs/no_tracker_demo_config.example.json \
  --repeat 3
```

16.7.3 成功条件

各 Tello について、command_all, battery_all, status_all がすべて ok になることを確認する。この段階では飛行しないため、最初に必ず実施する。

16.8 Demo 2: 3 機同時 takeoff-hover-land デモ

16.8.1 目的

Demo 2 は, 3 台の Tello が同時に `takeoff` し, メイン PC から周期的に `rc 0 0 0 0` を受け取り, 最後に `land` できることを確認する. 外部計測がないため, 水平移動は行わない.

このデモで確認できるのは, 以下である.

- 3 台へほぼ同時に `takeoff` を送れる.
- メイン PC が 3 台へ周期的に `rc` 指令を送れる.
- Raspberry Pi 側 `watchdog` と衝突せずに, メイン PC から keep-alive 的に `rc 0 0 0 0` を送り続けられる.
- 最後に全機へ `land` を送れる.

16.8.2 実行コマンド

```
cd tello_no_tracker_demo_patch/python/no_tracker_demos
python demo_02_takeoff_hover_land.py \
  --config ../../configs/no_tracker_demo_config.example.json \
  --duration 8 \
  --rate-hz 10 \
  --yes
```

16.8.3 処理の流れ

1. 3 台の Raspberry Pi へ TCP 接続する.
2. 3 台へ `command` を送る.
3. 3 台へ `battery?` を送り, 応答を確認する.
4. 3 台へ `takeoff` を送る.
5. 数秒待つ, 離陸直後の安定化を待つ.
6. 指定時間だけ `rc 0 0 0 0` を周期送信する.
7. 最後に 3 台へ `land` を送る.

16.8.4 成功条件

すべての Tello が離陸し, 水平移動せずホバリングし, 最後に正常に着陸すれば成功である. ただし, Tello の内部制御や床面条件により多少のドリフトはあり得るため, 十分な間隔を確保して行う.

16.9 Demo 3: 役割付き同期 yaw デモ

16.9.1 目的

Demo 3 は, 3 台の Tello に異なる役割を割り当て, メイン PC が各機へ別々の `action` を周期配信できることを確認する. 外部計測なしで水平移動は危険なため, `yaw` のみを使う.

例として、3 台に次の役割を与える。

```
Tello-1: yaw positive
Tello-2: yaw negative
Tello-3: hover
```

16.9.2 実行コマンド

```
cd tello_no_tracker_demo_patch/python/no_tracker_demos
python demo_03_sync_yaw_roles.py \
  --config ../../configs/no_tracker_demo_config.example.json \
  --duration 10 \
  --yaw-norm 0.20 \
  --rate-hz 10 \
  --yes
```

16.9.3 action の意味

各 action は

```
[lr, fb, ud, yaw]
```

であり、Demo 3 では $lr=0$, $fb=0$, $ud=0$ とする。したがって、水平移動は指令しない。たとえば $rc_limit=25$, $yaw_norm=0.20$ の場合、yaw チャンネルにはおおよそ 5 または -5 が送られる。

16.9.4 成功条件

3 台が同時に離陸し、Tello-1 と Tello-2 が反対向きにゆっくり yaw 回転し、Tello-3 が hover することを確認する。このとき、ログには各 Tello へ送った action と gateway から返った rc コマンドが保存される。

16.10 Demo 4: 仮想イベントトリガ協調デモ

16.10.1 目的

Demo 4 は、外部計測なしでも、メイン PC 内で合意計算・イベントトリガ判定を行い、その結果を Tello への役割別 action として配信できることを確認する。

このデモでは、実 Tello の位置を使わない。代わりに、メイン PC 内に仮想的な状態 θ_i を持ち、リング近傍の合意更新に近い計算を行う。最後に送信した値 $\hat{\theta}_i$ との差がしきい値を超えたときだけ、イベント発火とみなす。

```
if |theta_i - theta_hat_i| >= epsilon_q:
    event_i = true
    theta_hat_i <- theta_i
else:
    event_i = false
```


イベントが発火した機体には短い yaw pulse を与え、発火しなかった機体は hover する。このデモは、方策・合意・イベントトリガ通信の流れを実機コマンド配信経路に接続するための準備である。

16.10.2 非飛行 dry-run

まずは非飛行で実行する。この場合、takeoff も rc 送信も行わず、仮想計算とログ保存だけを確認する。

```
cd tello_no_tracker_demo_patch/python/no_tracker_demos
python demo_04_virtual_event_demo.py \
  --config ../../configs/no_tracker_demo_config.example.json \
  --steps 60
```

16.10.3 飛行モード

dry-run でイベントログが確認できた後、必要に応じて飛行モードで実行する。この場合も水平移動は行わず、イベント結果を yaw pulse として反映するだけである。

```
cd tello_no_tracker_demo_patch/python/no_tracker_demos
python demo_04_virtual_event_demo.py \
  --config ../../configs/no_tracker_demo_config.example.json \
  --steps 60 \
  --fly \
  --yaw-norm 0.18 \
  --yes
```

16.10.4 成功条件

非飛行 dry-run では、各ステップの theta、イベント発火状況、action がログに保存されることを確認する。飛行モードでは、イベントが発火した機体だけが yaw pulse を行い、発火しない機体は hover することを確認する。

16.11 ログの確認

各デモは JSONL 形式でログを保存する。保存先は設定ファイルの log_dir、または実行時の --log-dir で指定する。1 行が 1 イベントであり、以下のような情報が含まれる。

- 壁時計時刻 t_wall
- イベント名 event
- 対象 Tello 名 drone
- 送信 action
- gateway からの応答
- エラー発生時の例外文字列
- Demo 4 では仮想状態 theta とイベント発火状況

ログの例を以下に示す。

```
{
  "t_wall": ...,
  "event": "battery_all",
  "drone": "tello-1",
  "ok": true,
  "reply": {
    "ok": true,
    "resp": "87"
  }
}
{
  "t_wall": ...,
  "event": "yaw_role_rc",
  "step": 12,
  "drone": "tello-2",
  "action": [0, 0, 0, -0.2],
  "reply": {
    "cmd": "rc 0 0 0 -5"
  }
}
```

16.12 外部計測なしデモの詳細実行手順

本節では、外部計測なしデモを実際に実行する際の具体的な順序をまとめる。ここでいう成功には二段階ある。第一段階はメイン PC から Raspberry Pi の gateway へ接続できることであり、第二段階は Raspberry Pi から対応する Tello へ SDK コマンドを送り、ok やバッテリー値などの応答を得られることである。Demo 1 では、この二つを分けて確認する。

16.12.1 Step 0: フォルダ構成の確認

no_tracker_demo_patch を既存の Tello-drone フォルダへ組み込んだ後の想定構成は次である。

```
Tello-drone/ |——
configs/ |
  |—— pc_real_config.local.json |
  |—— no_tracker_demo_config.local.json |
  |—— no_tracker_demo_config.example.json |——
python/ |
  |—— run_real_baseline.py |
  |—— tello_rl/ |
  |   |—— real/ |
  |       |—— pi_client.py |
  |       |—— real_tracker.py |
  |       |—— real_safety_shield.py |
  |       |—— real_tello_env.py |
  |—— no_tracker_demos/ |
  |   |—— no_tracker_common.py |
  |   |—— demo_01_comm_check.py |
  |   |—— demo_02_takeoff_hover_land.py |
  |   |—— demo_03_sync_yaw_roles.py |
  |   |—— demo_04_virtual_event_demo.py |——
raspberrypi/ |
  |—— tello_gateway.py |——
docs/ |
  |—— tello_no_tracker_demo_steps.tex |——
logs/
  |—— no_tracker_demos/
```

メイン PC 側のデモスクリプトは、通常は Tello-drone/python から実行する。例えば Demo 1 は次のように起動する。

```
cd C:\Users\inulab\Work\Tello-drone\python
python .\no_tracker_demos\demo_01_comm_check.py ^
```

```
--config ..\configs\no_tracker_demo_config.local.json ^  
--repeat 3
```

PowerShell では改行継続にバッククォート ` を使ってもよい. 1 行で実行する場合は次の形でよい.

```
python .\no_tracker_demos\demo_01_comm_check.py --config ..\configs\  
no_tracker_demo_config.local.json --repeat 3
```

16.12.2 Step 0.5: メイン PC と Raspberry Pi の Python 仮想環境を有効化する

メイン PC 側では, PowerShell で Tello-drone/python に移動し, 仮想環境を有効化する. Activate.ps1 が実行できない場合は, 実行ポリシーを CurrentUser または Process スコープで許可してから再実行する.

```
cd C:\Users\inulab\Work\Tello-drone\python  
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned  
.\venv\Scripts\Activate.ps1
```

各 Raspberry Pi 側では, メイン PC の PowerShell から SSH 接続し, SSH 先のシェルで仮想環境を有効化する. ユーザー名は全台で inulab である.

```
ssh inulab@192.168.50.101  
ssh inulab@192.168.50.102  
ssh inulab@192.168.50.103  
  
# 各 SSH 画面の中で実行  
source ~/venv/bin/activate
```

16.12.3 Step 1: 各 Raspberry Pi で Tello Wi-Fi 接続を確認する

各 Raspberry Pi は, 有線 LAN 側でメイン PC と接続しつつ, 無線 LAN 側で対応する Tello の Wi-Fi に接続している必要がある. 各 Raspberry Pi に SSH して, 次を確認する.

```
iwgetid  
hostname -I  
ip addr show wlan0  
ip addr show eth0  
ip route get 192.168.10.1
```

iwgetid で TELLO-XXXXXX のような SSID が表示されることを確認する. また, hostname -I または ip addr show eth0 で, 有線 LAN 側 IP が no_tracker_demo_config.local.json の host と一致していることを確認する.

必要であれば, Tello Wi-Fi へ接続し直す.

```
nmcli dev wifi list  
sudo nmcli dev wifi connect TELLO-XXXXXX ifname wlan0
```

16.12.4 Step 2: Raspberry Pi 単体で Tello SDK 応答を確認する

PC からのデモを実行する前に、各 Raspberry Pi から直接 Tello へ `command` と `battery?` を送って応答を確認する。この確認では、`tello_gateway.py` が起動していると UDP ポートが競合する可能性があるため、gateway をいったん停止してから実行する。

```
# gateway が起動している場合は Ctrl+C で停止する
python3 - <<'PY'
import socket, time

addr = ("192.168.10.1", 8889)
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind("", 8889)
sock.settimeout(5)

for cmd in ["command", "battery?"]:
    print("send:", cmd)
    sock.sendto(cmd.encode(), addr)
    try:
        data, src = sock.recvfrom(1024)
        print("reply:", data.decode(errors="replace"), "from", src)
    except TimeoutError:
        print("timeout: no reply")
    time.sleep(1)
PY
```

期待される出力は次である。

```
send: command
reply: ok from ('192.168.10.1', 8889)
send: battery?
reply: 87 from ('192.168.10.1', 8889)
```

ここで `timeout: no reply` になる場合は、PC 側プログラムではなく、Raspberry Pi-Tello 間の接続を先に修正する。典型的な原因は、Tello Wi-Fi に接続できていない、別の Tello に接続している、Tello の電源が入っていない、または 192.168.10.1 への経路が `wlan0` になっていないことである。

16.12.5 Step 3: 各 Raspberry Pi で gateway を起動したままにする

各 Raspberry Pi で `tello_gateway.py` を起動する。このプロセスは、メイン PC からの TCP 接続を待ち受け、受け取った JSON メッセージを Tello SDK コマンドへ変換する。

```
python3 tello_gateway.py \
--listen-host 0.0.0.0 \
--listen-port 10000 \
--auto-command
```

起動後、次のように表示され、シェルのプロンプトが戻ってこない状態が正常である。

```
[gateway] sending SDK command
ok
[gateway] listening on 0.0.0.0:10000
```

もし [gateway] listening ... の後にすぐプロンプトが戻る場合、gateway は終了している。その場合、別ターミナルで `ss` を確認しても待受ポートは表示されない。gateway が正常に常駐しているかは、別の SSH ターミナルで次のように確認する。

```
ss -ltnp | grep 10000
ps aux | grep tello_gateway
```

期待される表示は次である。

```
LISTEN 0 5 0.0.0.0:10000 0.0.0.0:* users:(("python3",pid=1234,fd=5))
```

16.12.6 Step 4: メイン PC から gateway の TCP ポートを確認する

メイン PC の PowerShell から、各 Raspberry Pi の 10000 番ポートに接続できるか確認する。

```
Test-NetConnection 192.168.50.101 -Port 10000
Test-NetConnection 192.168.50.102 -Port 10000
Test-NetConnection 192.168.50.103 -Port 10000
```

TcpTestSucceeded : True であれば、PC-Raspberry Pi 間の TCP 接続は成功している。ここまで通った状態で Demo 1 を実行する。

16.12.7 Step 5: Demo 1 を実行し、結果を二段階で解釈する

Demo 1 は、非飛行で `command`, `battery?`, `status` を送る通信確認である。

```
cd C:\Users\inulab\Work\Tello-drone\python
python .\no_tracker_demos\demo_01_comm_check.py --config ..\configs\
no_tracker_demo_config.local.json --repeat 3
```

`no_tracker_demo_config.local.json` の `pi_timeout_s` は、初期確認では短すぎない値にする。Tello からの SDK 応答が遅れることがあるため、まずは次のように 5 秒程度に設定する。

```
"pi_timeout_s": 5.0
```

理想的な出力は、`command` に対して `ok`, `battery?` に対してバッテリー値が返る状態である。

```
[command_all]
tello-1: ok elapsed=0.12s reply=ok
tello-2: ok elapsed=0.10s reply=ok
tello-3: ok elapsed=0.11s reply=ok

[battery_all]
tello-1: ok elapsed=0.08s reply=87
tello-2: ok elapsed=0.09s reply=91
tello-3: ok elapsed=0.08s reply=84
```

一方、次のように ok だが reply が空で, elapsed がほぼ 2 秒になっている場合がある.

```
[command_all]
tello-1: ok elapsed=2.002s reply=

[battery_all]
tello-1: ok elapsed=2.001s reply=

[status_all]
tello-1: ok reply={'sdk_mode': True, ..., 'last_tello_response': '', 'last_state':
None}
```

この場合, PC から Raspberry Pi gateway への接続は成功しているが, Raspberry Pi から Tello への SDK 応答は確認できていない. last_tello_response が空で, last_state が None の場合は, Step 2 の Raspberry Pi 単体テストへ戻り, Tello Wi-Fi 接続と SDK 応答を確認する.

したがって, Demo 1 の判定は次のように分ける.

- status が即座に戻る: PC-Raspberry Pi 間の TCP 通信は成立している.
- command の reply=ok が返る: Raspberry Pi-Tello 間の SDK command 通信も成立している.
- battery? の reply に数値が返る: Tello からの query 応答も取得できている.
- reply が空で 2 秒程度かかる: gateway は動いているが, Tello からの応答が返っていない可能性が高い.

16.12.8 Step 6: Demo 2 以降に進む条件

Demo 2 以降は飛行を伴うため, 最低限, 次の条件を満たしてから進む.

- 3 台すべてについて, PC から gateway の status が返る.
- 3 台すべてについて, Raspberry Pi 単体テストで command に ok が返る.
- 3 台すべてについて, battery? に数値が返り, バッテリ残量が十分である.
- 各 gateway が 0.0.0.0:10000 で待ち受けており, プロンプトが戻らず常駐している.
- 飛行領域が十分広く, 3 台の Tello が互いに十分離れている.

条件を満たしたら, まず短時間の hover デモだけを実行する.

```
python .\no_tracker_demos\demo_02_takeoff_hover_land.py --config ..\configs\
no_tracker_demo_config.local.json --hover-seconds 10
```

その後, 必要に応じて yaw のみを使う役割付きデモへ進む.

```
python .\no_tracker_demos\demo_03_sync_yaw_roles.py --config ..\configs\
no_tracker_demo_config.local.json --duration 15 --yaw-norm 0.15
```

最後に, 仮想イベントトリガ協調デモは, まず dry-run で実行する.

```
python .\no_tracker_demos\demo_04_virtual_event_demo.py --config ..\configs\
no_tracker_demo_config.local.json --steps 60
```

飛行モードで実行する場合でも、水平移動は指令せず、イベント発火を低速 yaw pulse として反映するだけにする。

16.13 推奨実行順序

外部計測なしの段階では、以下の順序で確認する。

1. Demo 1 を実行し、非飛行で 3 台分の通信が正常であることを確認する。
2. Demo 2 を短時間で実行し、3 台同時の離陸・hover・着陸を確認する。
3. Demo 3 を短時間で実行し、役割別 yaw action の配信を確認する。
4. Demo 4 を dry-run で実行し、仮想合意・イベントトリガのログを確認する。
5. 必要に応じて Demo 4 を飛行モードで実行し、イベント発火を yaw pulse として反映する。
6. 外部計測系を導入した後、位置フィードバック制御、複数機距離維持、対象物取り囲みへ進む。

16.14 本デモの限界

本デモで確認できるのは、通信・同期・役割別指令・watchdog・ログ保存までである。以下は確認できない。

- 対象物を中心とする取り囲み誤差。
- 機体間距離制約の実時間監視。
- 壁距離制約の実時間監視。
- 外部計測に基づく安全シールド。
- 学習済み方策の位置フィードバック性能。

したがって、論文や発表では、本デモを「外部計測導入前の通信・制御基盤確認デモ」と位置づけるのが適切である。協調取り囲み制御の実証は、外部計測系を導入して各 Tello と対象物の位置を取得できるようになってから行う。

16.15 トラブルシューティング

16.15.1 Demo 1 で connection refused になる

Raspberry Pi 側で `tello_gateway.py` が起動していない、または IP アドレス・ポートが設定ファイルと一致していない可能性がある。各 Raspberry Pi 上で gateway の起動ログを確認する。

16.15.2 battery? が空文字になる

Raspberry Pi は Tello Wi-Fi に接続できているが、Tello が SDK mode に入っていない可能性がある。--auto-command を付けて gateway を起動するか、Demo 1 で --skip-command を付けずに実行する。

16.15.3 takeoff 後にすぐ land する

Raspberry Pi 側 watchdog が、PC からの rc 指令欠落を検出している可能性がある。Demo 2 では `rate_hz` を 10 Hz 程度にし、PC と Raspberry Pi 間の有線 LAN 接続が安定しているか確認する。

16.15.4 yaw が速すぎる

`--yaw-norm` または `--rc-limit` を下げる。初期値としては、`rc_limit=25`, `yaw_norm=0.15` から始めるとよい。

16.15.5 機体が水平に流れる

外部計測なしではドリフトを補正できない。即座に land し、飛行領域、床面、照明、Tello の状態、バッテリー残量を確認する。本デモでは水平移動を指令していないため、流れが大きい場合は飛行デモを中止する。

17 まとめ

本追加実装では、通常 Tello の Wi-Fi 制約を回避するために、Tello 1 台につき Raspberry Pi 1 台を割り当てるゲートウェイ構成を採用する。PC は全体の状態推定、安全評価、制御計算を担当し、Raspberry Pi は対応する Tello への SDK コマンド送信を担当する。この分担により、既存の command-level action 表現を保ったまま、通常 Tello 複数台の実機実験へ移行できる。

実機投入は、必ず単機通信確認、外部計測確認、単機位置制御、複数機固定位置制御、静止対象物取り囲み、移動対象物追従、学習済み方策評価の順に進める。特に、外部計測と安全シールドが安定するまでは、RL 方策を直接実機へ接続しない。